

UNIVERSIDAD GALILEO
Postgrado en Diseño y Desarrollo de Software

INFRAESTRUCTURA EN LA NUBE
Proyecto de Curso

Sistema de Reservas y Disponibilidad de Parqueos

Entrega 1

INTEGRANTES

Ana Gabriela López Ortíz 18002988

Pablo Samuel Siquinajay Castro 22006477

Christian Javier Sosa Ruiz 22000102

Ciudad de Guatemala, 17 de mayo de 2026

ENTREGA 1

Resumen ejecutivo

Descripción del problema

Actualmente, muchas empresas cuentan con una cantidad limitada de espacios de parqueo en comparación con el número total de colaboradores. Con la implementación de modelos de trabajo híbrido (presencial y teletrabajo), la asistencia de empleados varía diariamente, lo que dificulta administrar de manera eficiente la disponibilidad de parqueos corporativos.

La falta de un sistema centralizado provoca conflictos de asignación, espacios desaprovechados y procesos manuales poco eficientes para gestionar reservas y disponibilidad.

Por ello, se propone desarrollar un Sistema de Reserva y Gestión de Parqueos Empresariales que permita a las empresas administrar de forma automatizada la disponibilidad de espacios para sus colaboradores. El sistema permitirá realizar reservas, visualizar disponibilidad en tiempo real, evitar conflictos de asignación y optimizar el uso de los parqueos disponibles, mejorando así la gestión interna y la experiencia de los colaboradores.

Actores

Actores Primarios

- **Usuario/Conductor:** Es la persona registrada en el sistema, la cual consulta disponibilidad, solicita y confirma reservas de espacios de parqueo.
- **Administrador:** Gestiona el inventario de espacios y consulta reportes de ocupación y asistencia, crea y edita características del parqueo.

Actores de Soporte

- **Servicio de autenticación:** Gestiona el registro e inicio de sesión de usuarios, controlando roles (Usuario o Administrador).
- **Servicio de almacenamiento en nube:** Persisten los comprobantes digitales generados tras cada reserva confirmada.
- **Servicio de mensajería asíncrona:** Ejecuta la liberación automática de espacios no confirmados tras 20 minutos y dispara las alertas de confirmación al usuario.
- **Servicio de notificaciones:** Envía alertas y confirmaciones de reserva al conductor (correo electrónico).

Casos de uso priorizados

1. Inicio de sesión

Como empleado, debo de acceder con un usuario previamente creado por el administrador e iniciar sesión para poder acceder al sistema de reservas.

Prioridad: Po

Criterio de éxito: El empleado inicia sesión con las credenciales asignadas por el administrador. El sistema emite un token de sesión válido con el rol correspondiente. Credenciales incorrectas retornan HTTP 401.

2. Consulta de disponibilidad en tiempo real

Como conductor, quiero consultar la disponibilidad de espacios filtrada por tipo de vehículo para saber si hay lugar antes de reservar.

Prioridad: Po

Criterio de éxito: El endpoint `GET /availability` retorna el estado de cada espacio (Disponible/Reservado/Ocupado) en tiempo real, respetando restricciones de rol del conductor.

3. Reserva de espacio

Como conductor, quiero reservar un espacio y recibir un comprobante digital para garantizar mi lugar.

Prioridad: Po

Criterio de éxito: El endpoint `POST /reservar` asigna el espacio atómicamente previniendo double-booking. El conductor recibe un comprobante QR en PDF por correo en menos de 30 segundos.

4. Gestor de usuarios para administrador

Como administrador, quiero gestionar los colaboradores registrados en el sistema, incluyendo crear, editar y desactivar cuentas y asignar roles. Controlar qué usuarios pueden reservar y qué tipo de espacios tienen permitido ocupar.

Prioridad: Po

Criterio de éxito: El administrador crea, edita y desactiva cuentas asignando roles (operativo, ejecutivo, visitante frecuente). Un usuario desactivado pierde acceso inmediatamente pero conserva su historial. Ningún colaborador puede modificar su propio rol ni el de terceros.

5. Gestor de parqueos para administrador

Como administrador, quiero configurar los espacios de parqueo disponibles, definiendo qué tipo de vehículo es permitido y qué roles de colaborador pueden ocuparlos, para que el sistema asigne reservas respetando las restricciones operativas de la empresa.

Prioridad: Po

Criterio de éxito: El administrador crea y edita espacios asignando identificador, tipo de vehículo y roles autorizados. Puede marcar espacios como inactivos por rango de fechas, quedando invisibles para reserva. Un colaborador con rol no autorizado no puede reservar un espacio restringido aunque esté disponible.

6. Liberación automática por no confirmación

Como sistema, quiero liberar automáticamente espacios no confirmados para optimizar la disponibilidad.

Prioridad: P1

Criterio de éxito: Si el conductor no confirma en 20 minutos, el espacio vuelve a Disponible de forma asíncrona y se notifica al conductor antes de la liberación.

7. Dashboard de ocupación para administrador

Como administrador, quiero visualizar un dashboard de ocupación para monitorear el estado del parqueo en tiempo real.

Prioridad: P2

Criterio de éxito: El dashboard presenta de forma gráfica el conteo de espacios en cada estado (Disponible, Reservado, Ocupado) actualizado en intervalos no mayores a 30 segundos. El administrador puede distinguir la ocupación por tipo de vehículo (moto/carro) sin necesidad de consultas adicionales.

8. Gestión de tarifas

Como administrador, quiero establecer precios diferenciados por tipo de vehículo.

Prioridad: P2

Criterio de éxito: El administrador define y actualiza tarifas para moto y carro. El cambio es inmediato y queda registrado con marca de tiempo para auditoría.

Funcionalidades específicas del proyecto:

1. Reserva con validación de placa registrada

El sistema solo permite reservar si la placa del vehículo está previamente registrada en el perfil del colaborador. No cualquier persona con cuenta puede reservar — tiene que haber un vehículo vinculado y aprobado. Un colaborador puede tener más de una placa (ej. carro y moto), pero solo puede tener una reserva activa por día.

2. Asignación de tipo de espacio por perfil de colaborador

El Admin puede categorizar colaboradores: ejecutivo, operativo, visitante frecuente. Cada categoría tiene acceso a un pool de espacios distinto. Un colaborador operativo no puede reservar un espacio marcado como ejecutivo aunque esté disponible. Esto refleja jerarquías reales que las empresas ya manejan manualmente hoy.

3. Historial de asistencia vehicular por colaborador

Dado que cada reserva está vinculada a un colaborador identificado, el sistema genera automáticamente un registro de qué días llegó en vehículo, a qué hora confirmó su llegada y cuánto tiempo ocupó el espacio. El admin puede exportar esto. Es un diferenciador fuerte porque las empresas hoy no tienen visibilidad de esto.

4. Liberación anticipada voluntaria

El colaborador puede liberar su espacio antes de la hora de salida desde la app (ej. salió temprano). Esto actualiza disponibilidad en tiempo real y permite que otro colaborador tome ese espacio el mismo día. Es distinto al timeout automático de 20 minutos — esta es una acción explícita del usuario que beneficia al resto.

5. Política de cancelación con penalización configurable

Si un colaborador cancela una reserva menos de N horas antes (configurable por el admin), el sistema lo registra como "cancelación tardía". Acumular X cancelaciones tardías en un mes puede bloquear temporalmente la capacidad de reservar. Esto desincentiva el comportamiento de reservar por precaución sin intención real de usar el espacio.

Mockups del FE

Se diseñaron 7 pantallas low-fi que cubren los casos de uso priorizados (P0, P1 y P2). Los wireframes completos se encuentran en el Anexo A.

Mapeo a conceptos del curso

Esta sección vincula las necesidades operativas de nuestro sistema de parqueos con el stack tecnológico del curso, vincula los componentes con las funcionalidades específicas que nuestro Sistema de Reservas y Disponibilidad de Parqueos va a hacer.

Componente del curso	Cómo lo ejercita su proyecto
Cómputo (API)	- Endpoint <code>GET/availability</code> consulta disponibilidad de parqueo filtrando por tipo de vehículo (moto, carro) del lado del cliente. - Endpoint <code>POST/reservar</code> ejecuta la lógica de validación de disponibilidad y asigna espacios en ese mismo instante.
Base de datos	Inventario de espacios con estados <i>Libre/Ocupado</i> y almacenamiento de datos de usuario registrados(perfil de datos) + sus placas vehiculares.

Almacenamiento de archivos	• Comprobantes digitales(en PDF/PNG) de finalización de reserva del parqueo. • Hosting de archivos estáticos; como imágenes de referencia de las entradas de los parqueos. Ayuda visual para el conductor.
Procesamiento asíncrono	• Si el usuario no confirma digitalmente su ingreso tras 20 minutos de la hora pactada, un <i>Worker</i> asíncrono procesa la cancelación, libera el cupo en la base de datos y lo marca el espacio de “Reservado” a “Disponible”.
Red	• El sistema expone únicamente el API Gateway a internet público vía HTTPS. Los servicios internos (base de datos, worker de liberación asíncrona) residen en subredes privadas sin acceso directo desde el exterior.
Seguridad	• Roles de IAM ; El "<i>Admin</i>" solo puede ver reportes y gestionar; el "<i>User</i>" solo genera reservas. • Cifrado de datos sensibles (ej. <code>license_plate_number_user</code> y <code>user_phone_number</code>).
Observabilidad	• <i>Alarma de saturación: notificación automática si la disponibilidad de espacios llega a cero</i>

Scope (in/out)

Declaración explícita de qué hará el sistema y qué queda fuera del diseño.

- **In:**
 - Gestión de identidad(Auth); Para el registro de usuarios al sistema. Mejora el control de usuarios que ingresan al sistema y los permisos que tiene(User/Admin)
 - Búsqueda de espacios disponibles; Muestra los espacios disponibles de parqueo de la empresa.
 - Gestión automática de reservas; Mediante una reserva de espacio en tiempo real que asegure atomicidad de espacio(ej. para que 2 o más personas no reserven el mismo espacio al mismo tiempo). El sistema rechaza a uno y rechaza al otro por orden de reserva.
 - Liberación asíncrona; Cancelación automática por no confirmación el día programado (asíncrono), con una espera máxima de 20 minutos y alertas de confirmación, entonces, liberar espacio antes ocupado.
 - Generación de comprobantes de confirmación de reservas; Generación de comprobante único digital e inalterable para el acceso. El sistema genera un QR en PDF que se guarda en el almacenamiento de la nube.
 - Dashboard interactivos de Disponibilidad/Ocupación;

- Un cliente podrá
 - Consultar disponibles(tendrá mensajes en la UI de “Ocupado”/”Disponible”),
 - Solicitar reserva, confirmar reserva.
 - Marcar en la app un botón de “Ocupar Parqueo”, y el estado en la BD cambia a "Ocupado"
- Para el Admin, podrá ver
 - En su inventario(mediante algún tipo de gráfico) los espacios disponibles, espacios reservados y actualmente ocupados.
 - Establecer el precio del parqueo por moto y carro.
 - Que usuarios han reservado y que días (brinda control de asistencia)
 - Respecto a la disponibilidad del parqueo, se usarán cantidad de parqueos disponibles(por motivos prácticos y sencillez) para ubicar qué número de parqueo es el que se encuentra como “Ocupado”/”Disponible”.
- **Out:**
 - Integración física con hardware.
 - Procesamiento de pagos reales; Por tema de complejidad, y costos.
 - **App Móvil Nativa**

Preguntas abiertas E1

¿Qué mecanismo concreto garantiza la atomicidad en la asignación de espacios? Se menciona que "el sistema rechaza a uno por orden de reserva", pero no se especifica si se usará locking optimista, pesimista, transacciones ACID, o algún mecanismo de cola (queue).

¿Cómo se maneja el race condition a nivel de base de datos cuando dos requests llegan en el mismo milisegundo?

¿Qué tecnología ejecuta la liberación automática a los 20 minutos? No se define si es un cron job, un mensaje en una cola (SQS, RabbitMQ), un scheduled function (Cloud Scheduler, Lambda EventBridge), o un timer en base de datos. Cada opción tiene implicaciones de confiabilidad distintas.

¿Cómo integraremos una función más eficaz cuando el cliente llegue “verdaderamente” al parqueo más que solo un clic de “Ocupar parqueo”? ¿Un IoT como sensor de parqueo sería suficiente?

Anexos

Anexo A: Mockups

Pantalla 1 - Registro e inicio de sesión: Esta pantalla cubre el acceso al sistema. Se presentan dos vistas: el formulario de inicio de sesión para usuarios existentes y el formulario de registro para nuevos usuarios.

The mockup shows two side-by-side white cards on a light beige background. The left card is for login, featuring a 'P' icon, the title 'ParkReserva', and a sub-header 'Iniciar sesión'. It includes input fields for 'Correo electrónico' (with 'usuario@correo.com') and 'Contraseña' (masked with dots), an 'Ingresar' button, and a link '¿No tienes cuenta? Regístrate'. The right card is for registration, titled 'Crear cuenta' with the sub-header 'Completa tus datos'. It has input fields for 'Nombre' (Juan), 'Apellido' (Pérez), 'Correo electrónico' (usuario@correo.com), 'Tipo de vehículo' (Carro), 'Placa' (P-123ABC), and 'Contraseña' (masked). A 'Crear cuenta' button is at the bottom.

Pantalla 2 - Consulta de disponibilidad en tiempo real: Esta pantalla permite al conductor visualizar el estado actual de los espacios de parqueo. Se presenta una cuadrícula con cada espacio identificado por número, mostrando su estado mediante código de color: verde (Disponible), rojo (Ocupado) y amarillo (Reservado). El conductor puede filtrar por tipo de vehículo (moto/carro) antes de proceder a reservar.

The mockup displays a 'Disponibilidad de espacios' screen. At the top, it says 'Actualizado hace 3 segundos' and has filters for 'Carro' and 'Moto'. A legend shows green for 'Disponible', yellow for 'Reservado', and red for 'Ocupado'. Below is a 2x5 grid of parking spaces, each with a car icon and a number (A-01 to B-05). The colors indicate their status: A-01, A-02, B-02, B-04 are green; A-03, B-01, B-05 are red; A-04, B-03 are yellow. At the bottom, a summary shows 5 Disponibles, 2 Reservados, and 3 Ocupados, with a 'Reservar espacio' button.

Pantalla 3 - Flujo de reserva: Guía al conductor a través del proceso de reserva de un espacio seleccionado. Se muestran los detalles del espacio elegido, los datos del vehículo registrado y la hora estimada de llegada. El sistema valida la disponibilidad de forma atómica al momento de confirmar, previniendo el double-booking. Al completarse, se genera el comprobante QR.

Confirmar reserva
Revisa los detalles antes de continuar

Espacio seleccionado

 **Espacio A-02**
Zona A - Nivel 1 Disponible

Datos del vehículo

Tipo: **Carro** Placa: **P-123ABC**

Hora estimada de llegada
08:30 AM ⓘ

El espacio se liberará automáticamente si no es confirmado en 20 minutos.

Cancelar **Confirmar reserva**

Pantalla 4 - Comprobante de reserva con código QR: Una vez confirmada la reserva, el sistema genera y presenta al conductor un comprobante digital único con código QR. Este comprobante queda disponible para descarga inmediata en formato PDF y es enviado al correo registrado del conductor. El código QR es inalterable y sirve como credencial de acceso al parqueo.



Reserva confirmada
Tu comprobante ha sido generado

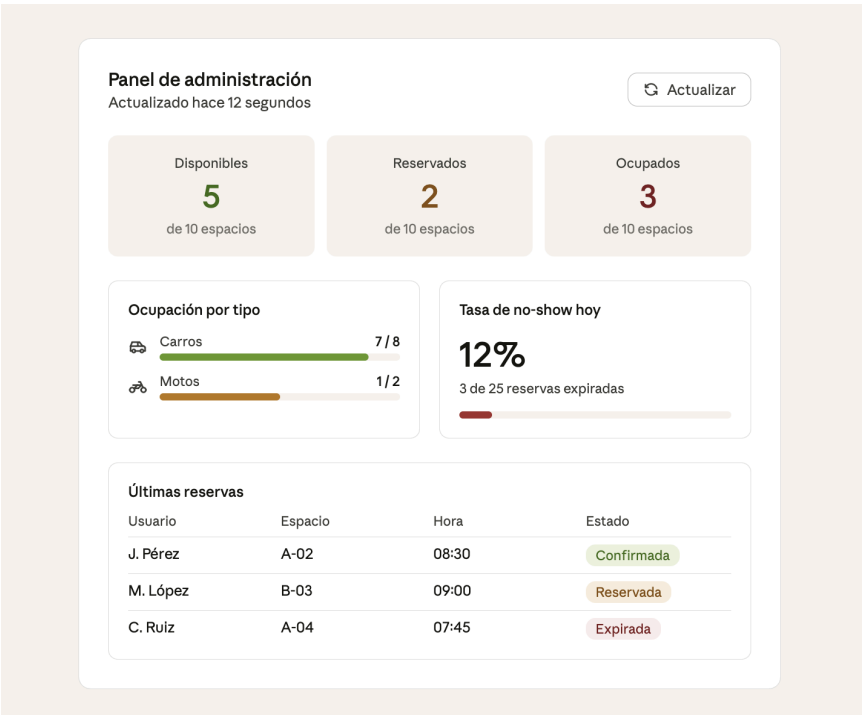
No. de reserva	#RES-20260515-0042
Espacio asignado	A-02 - Zona A
Placa	P-123ABC
Hora de llegada	08:30 AM - 15 may 2026



Presenta este QR al ingresar al parqueo

 Descargar PDF  Enviar correo

Pantalla 5 - Dashboard del administrador: Esta pantalla provee al administrador una visión general en tiempo real del estado del parqueo. Se presentan métricas de ocupación mediante gráficos y contadores por estado, diferenciados por tipo de vehículo.



Pantalla 6 - Gestión de precios por tipo de vehículo: Esta pantalla permite al administrador configurar y actualizar las tarifas diferenciadas por tipo de vehículo. Cada cambio queda registrado en el historial, para fines de auditoría.

Gestión de tarifas
Configura el precio por tipo de vehículo

Carro
Tarifa por hora (Q)
Q 10.00

Moto
Tarifa por hora (Q)
Q 5.00

Historial de cambios

Fecha	Tipo	Tarifa
15 may, 08:00	Carro	Q 10.00
10 may, 14:30	Moto	Q 5.00
01 may, 09:15	Carro	Q 8.00

Cancelar

Guardar cambios

Pantalla 7- Historial de reservas por usuario y fecha: Esta pantalla permite al administrador consultar y auditar el registro completo de reservas del sistema. La tabla es filtrable por usuario y rango de fechas, mostrando para cada entrada el espacio asignado, tipo de vehículo, estado final y marca de tiempo. Los registros son inmutables; ningún rol puede eliminarlos.

Usuario	Espacio	Vehículo	Fecha / Hora	Estado
Juan Pérez	A-02	Carro	15 may · 08:30	Confirmada
María López	B-03	Moto	15 may · 09:00	Reservada
Carlos Ruiz	A-04	Carro	14 may · 07:45	Expirada
Ana González	B-01	Moto	13 may · 10:15	Cancelada
Juan Pérez	A-01	Carro	12 may · 08:00	Confirmada

Anexo B — Anexo de IA

1. Prompt #1

"Actúa como un Arquitecto de Soluciones Cloud. Tengo un sistema de reserva de parqueos en Guatemala para un proyecto. Ayúdame a:

- 1. Redactar el cierre del Resumen Ejecutivo enfocándose en qué procesos se van a automatizar (evitar double-booking, liberación de espacios).*
- 2. Generar ejemplos específicos para una tabla que mapee las funcionalidades del sistema con los 7 componentes obligatorios del curso (Cómputo, DB, Almacenamiento, Red, Asíncrono, Seguridad, Observabilidad).*
- 3. Asegúrate de que los ejemplos no sean genéricos (no digas solo 'hacer login') sino que mencionen procesos reales de un parqueo inteligente."*

Nuestra reflexión

- **Qué aceptamos:** La estructura de la tabla y los ejemplos de procesos asíncronos, ya que es un "trade-off" interesante para el negocio.
- **Qué editamos:** Editamos ejemplos de procesos asíncronos, seguridad, red, observabilidad, Almacenamiento de archivos.
- **Qué descartamos:** Sugerencias de la IA sobre algunas funciones de red, como aislamiento de la base de datos y la lógica de negocio.

2. Prompt #2

"Actúa como Product Owner y genera una lista de Scope In/Out para un sistema de parqueos inteligente enfocado en un curso de Infraestructura Cloud, priorizando funcionalidades que usen bases de datos y colas de mensajes."

Reflexión del equipo

- **Qué aceptamos:** La estructura de "In/Out" sugerida por la IA, especialmente la idea de dejar fuera la integración física (sensores) y el desarrollo para una app móvil para no complicar el alcance técnico del curso.
- **Qué editamos:** Todas las sugerencias, ajustándose a un desarrollo más práctico y sencillo.
- **Qué descartamos:** La IA sugirió incluir "Machine Learning para predicción de demanda"; lo movimos a *Out-of-Scope* inmediatamente para mantener el proyecto ejecutable en 3 meses.

ENTREGA 2

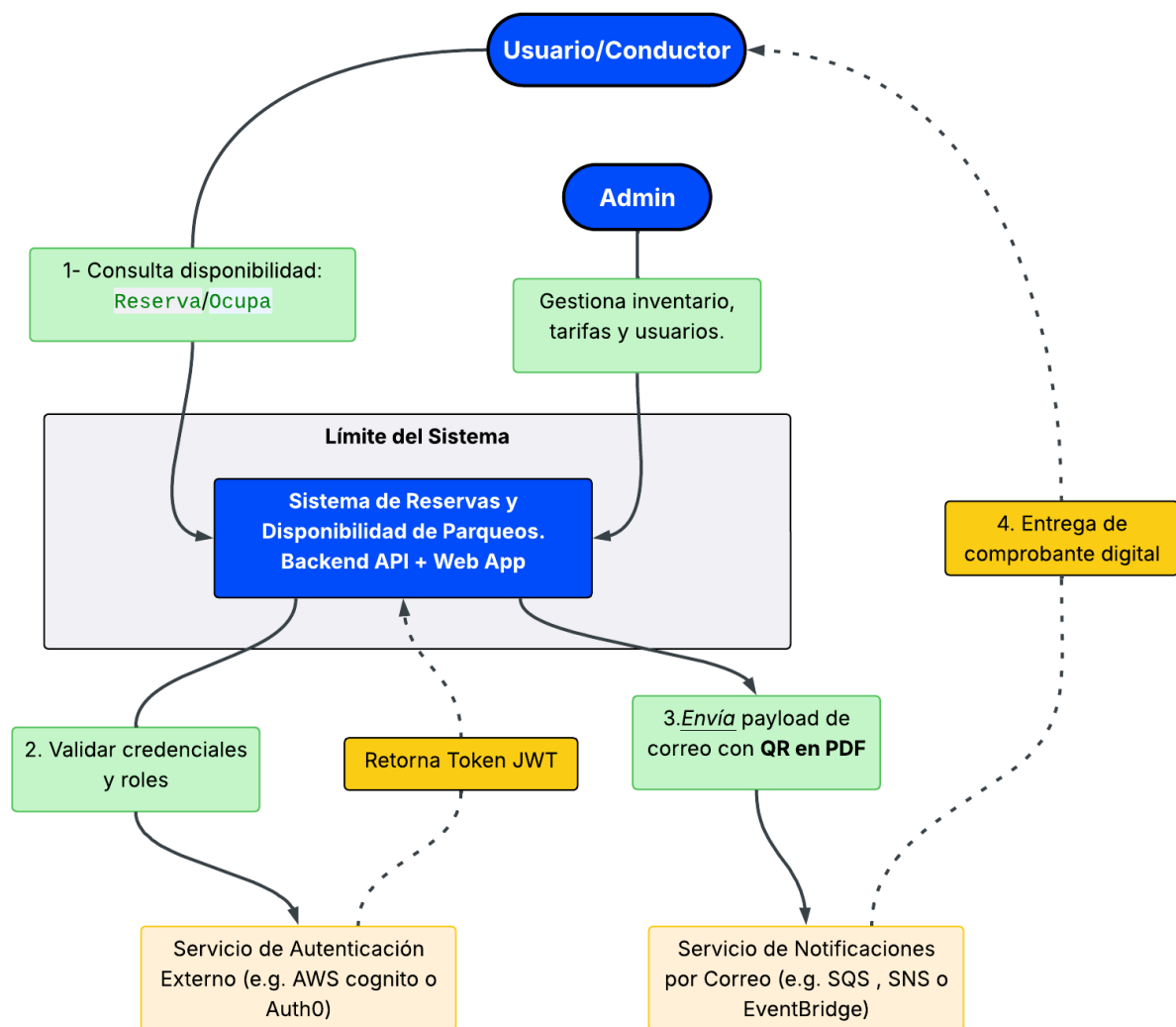
Resumen de cambios

- **Definición de la Estrategia de Concurrencia (Atomicidad):**

Se resolvió la duda de la *Entrega 1* sobre el mecanismo para evitar el *double-booking*. Se descartó la necesidad de implementar colas de mensajes bloqueantes o bloqueos pesimistas a nivel de aplicación; en su lugar, se optó por delegar la atomicidad a la base de datos mediante **índices únicos parciales** en PostgreSQL, simplificando la capa de cómputo.

Diagrama de contexto

El siguiente diagrama delimita las fronteras del Sistema de Reserva y Gestión de Parques Empresariales, también, definiendo los actores externos y sistemas de terceros.



Descripción de las Entidades del Diagrama

A. Límite del Sistema (System Boundary)

- **Sistema de Reservas y Disponibilidad de Parques:** Comprende todo el software desarrollado por el equipo (interfaces web de usuario/administrador y el backend API de funciones). Se encarga de procesar la lógica de negocio, calcular la disponibilidad en tiempo real, derivar estados vehiculares y gobernar la persistencia de datos relacionales y punteros de archivos.

B. Actores Externos (Usuarios Primarios)

- **Usuario/Conductor:** Interactúa con el sistema para consultar la disponibilidad de espacios en tiempo real, enviar solicitudes de reserva, registrar voluntariamente la ocupación o liberación anticipada y recibir sus comprobantes digitales.
- **Administrador (Personal de operaciones de la empresa):** Interactúa con el sistema para dar de alta o baja cuentas de colaboradores, configurar restricciones vehiculares por perfil(solo moto o carro) y auditar el historial de asistencia vehicular de la corporación.

C. Sistemas Externos de Terceros (Sistemas de Soporte)

- **Servicio de Autenticación Externo (e.g., AWS Cognito / Auth0):** Sistema gestionado externo encargado de validar las credenciales de acceso al sistema. Le quita la responsabilidad al sistema de almacenar contraseñas, retornando tokens criptográficos (JWT) que contienen el rol (`system_role`) del usuario para la posterior autorización en el API.
- **Servicio de Notificaciones (e.g., AWS SQS/SNS/Event Bridge):** Plataforma externa de entrega de correos electrónicos. Recibe las instrucciones del sistema transaccional de parques junto con el archivo PDF/PNG del comprobante generado, y se encarga de enviarlo directamente al buzón del conductor final.

Decisión de cómputo

Servicio elegido: AWS ECS Fargate

El backend del sistema se desplegará como un contenedor Docker gestionado por ECS Fargate. Fargate es la modalidad serverless de contenedores en AWS: el equipo define la task definition (imagen, CPU, memoria, puertos) y AWS gestiona el servidor, el sistema operativo y los parches. No se administra ninguna instancia EC2 directamente.

Por qué Fargate sobre las alternativas

AWS Lambda fue descartado porque el sistema expone una API REST con múltiples endpoints que necesitan estar disponibles de forma continua. Lambda está diseñado para ejecuciones cortas y puntuales y no es el modelo adecuado para un servidor HTTP

persistente con conexiones a base de datos que debe responder en tiempo real a consultas de disponibilidad cada 30 segundos.

EC2 con Auto Scaling fue descartado porque requiere administrar el servidor subyacente: actualizaciones del sistema operativo, parches de seguridad, configuración del runtime y gestión del ciclo de vida de las instancias. Para un equipo de tres personas en un proyecto de curso, ese overhead operativo no agrega valor al diseño del sistema.

Trade-offs

1. Disponibilidad continua vs. costo en baja demanda

Fargate mantiene el contenedor corriendo aunque no haya tráfico, lo que genera costo incluso en horarios de baja actividad (noches, fines de semana). Lambda cobraría solo por invocación y resultaría más económico para un sistema con uso muy intermitente. Para este sistema, donde el dashboard del administrador se refresca cada 30 segundos durante el horario laboral y las reservas ocurren en ventanas predecibles de la mañana, la disponibilidad continua está justificada: el costo de una respuesta lenta o un cold start en el momento pico de reservas (07:00–09:00) supera el ahorro nocturno.

2. Sin servidor que administrar vs. menor granularidad de control.

Al delegar la gestión del servidor a AWS, el equipo gana velocidad operativa pero pierde control sobre la configuración del sistema operativo, la red a nivel de host y la capacidad de instalar dependencias del sistema directamente. Para el backend de este proyecto, una API REST con dependencias definidas y empaquetadas en un contenedor, ese nivel de control no es necesario. Si en el futuro el sistema requiriera acceso a hardware especializado o configuraciones de red avanzadas a nivel de instancia, EC2 sería la opción correcta.

Desventaja reconocida

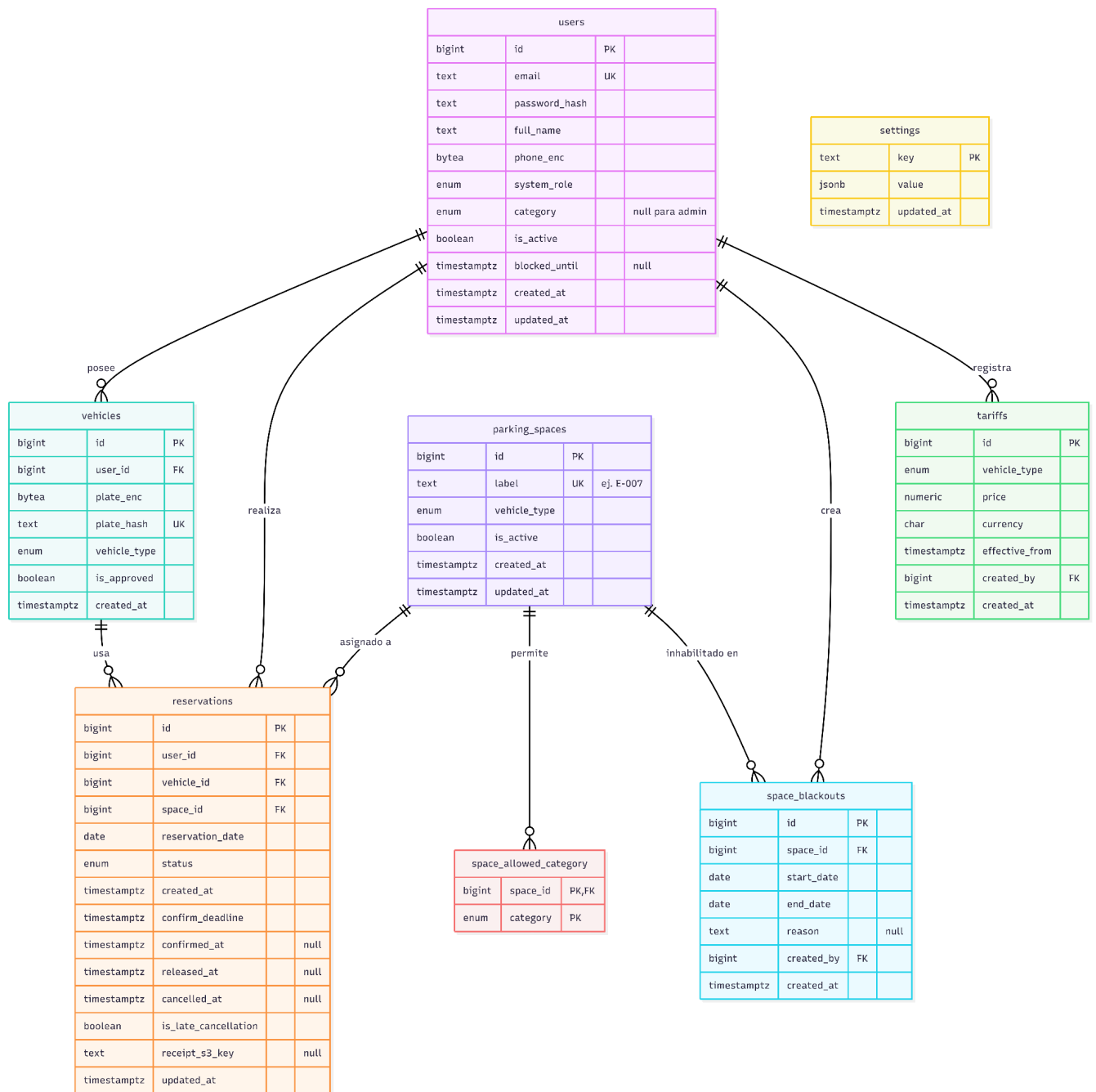
Fargate tiene una latencia de arranque de 20 a 30 segundos cuando escala desde cero tareas. Si el servicio se detiene por inactividad y recibe una solicitud en ese instante, la primera respuesta se verá afectada. Para el MVP se acepta este riesgo manteniendo al menos una tarea corriendo en todo momento, lo que elimina el cold start a costa de no escalar a cero. En producción, esto se mitigaría con una política de Auto Scaling que nunca baje de una tarea mínima.

Modelo de datos

El dominio se modela como un esquema relacional sobre PostgreSQL 16 (RDS), en modo single-company (un despliegue para una empresa, sin tabla de tenants). Se eligió un modelo relacional porque el negocio gira en torno a invariantes transaccionales (no permitir doble reserva, una reserva activa por día) y la integridad referencial entre colaboradores, vehículos, espacios y reservas, estas son propiedades que una base relacional garantiza de forma nativa.

Entidades principales

Entidad	Rol en el dominio
users	Colaboradores y administradores en una sola tabla, diferenciados por <code>system_role</code> .
vehicles	Vehículos del colaborador (1:N). Reservar exige una placa registrada y aprobada.
parking_spaces	Inventario de espacios; cada uno con un tipo de vehículo admitido.
space_allowed_category	Relación M:N que define qué categorías de colaborador pueden usar cada espacio.
space_blackouts	Inhabilitación de un espacio por rango de fechas.
reservations	Núcleo del modelo: sostiene disponibilidad, atomicidad, liberación, asistencia y el comprobante.
tariffs	Historial de precios append-only (auditoría); el precio vigente es la fila más reciente por tipo.
settings	Política configurable por el admin (ventana de cancelación, máximo de cancelaciones tardías).



Tres principios sostienen la coherencia del modelo

1. El estado del espacio (**Disponible/Reservado/Ocupado**) se *deriva*, no se **almacena**. Se calcula por (**espacio, fecha**) a partir de la reserva activa. Una columna **estado** fija no puede expresar "ocupado hoy, libre mañana" y se desincronizaría con las reservas.
2. **Atomicidad por índice único parcial**, no por bloqueos de aplicación (ver patrón de acceso 2).
3. **Dos ejes de "rol" separados**: **system_role** (admin/driver, para permisos) y **category** (ejecutivo/operativo/visitante_frecuente, para el pool de espacios accesible). No se fusionan.

Patrones de acceso principales

#	Patrón de acceso	Tipo	Frecuencia	Estructura que lo soporta
1	Consultar disponibilidad (GET /availability), filtrada por tipo de vehículo y categoría del conductor	Lectura	Muy alta	LEFT JOIN parking_spaces ↔ reservations del día activo; índice (reservation_date, status); exclusión de inactivos y space_blackouts.
2	Crear reserva (POST /reservar), atómica	Escritura (alta contención)	Alta	Índices únicos parciales (space_id, reservation_date) y (user_id, reservation_date) con WHERE status IN ('reservada','ocupada').
3	Confirmar llegada / liberar (ocupar, salida, liberación anticipada)	Escritura	Media	UPDATE reservations de status + marcas de tiempo (confirmed_at, released_at).
4	Liberación automática a los 20 min (worker P1)	Lectura + escritura batch	Periódica	Índice (status, confirm_deadline) para escanear reservas por expirar.
5	Historial de asistencia por colaborador (export del admin)	Lectura	Baja	Índice (user_id, reservation_date); campos confirmed_at/released_at.
6	Conteo de cancelaciones tardías del mes (penalización Fn5)	Lectura	Baja	is_late_cancellation + índice (user_id, reservation_date).
7	Precio vigente / historial de tarifas	Lectura	Baja	tariffs append-only + vista current_tariffs (DISTINCT ON (vehicle_type)).
8	Gestión (CRUD) de usuarios y espacios	Escritura	Baja	Claves primarias e índices únicos (email, label, plate_hash).

El patrón #1 es el más frecuente (la pantalla de disponibilidad se refresca cada ≤ 30 s) y por eso se optimiza con un índice dedicado y un conjunto de datos pequeño (decenas de espacios). El patrón #2 es el más crítico en correctitud: la **atomicidad la garantiza la base de datos**. Dos reservas simultáneas para el mismo espacio/día ejecutan **INSERT** concurrentes; el índice único parcial deja pasar solo uno y el otro recibe una violación de unicidad que el API traduce a **HTTP 409**. Es ACID, sin bloqueo pesimista ni cola de mensajes, responde la pregunta de E1 sobre cómo se evita el double-booking.

Qué va en base de datos vs. almacenamiento de objetos

Dato	Dónde	Justificación
Usuarios, vehículos, espacios, reservas, tarifas, settings	PostgreSQL (RDS)	Datos estructurados y relacionales, registros pequeños, requieren transacciones (reserva atómica), integridad referencial (FKs) y consultas indexadas (disponibilidad, historial).
Comprobantes de reserva (QR en PDF/PNG)	S3 (object storage)	Blobs binarios, inmutables (write-once), de tamaño medio, que se sirven por descarga/URL prefirmada. No requieren consultas relacionales; almacenarlos en BD inflaría el tamaño y degradaría backups.
Imágenes estáticas de referencia (entradas del parqueo)	S3	Contenido estático servido directamente; no es dato transaccional.
Puntero al comprobante (<code>reservations.receipt_s3_key</code>)	PostgreSQL	La BD guarda solo la clave/URL del objeto en S3, no el binario. Mantiene la fila ligera y enlaza el dato relacional con el objeto.

Decisión para MVP: no se introduce una capa de caché dedicada (ej. Redis/ElastiCache). La consulta de disponibilidad (patrón #1) opera sobre un conjunto pequeño (decenas de espacios) con un índice apropiado, por lo que PostgreSQL cumple holgadamente los SLAs (< 2 s de respuesta, reflejo del cambio en ≤ 30 s) sin caché. Añadir una caché ahora introduciría un problema de **invalidación y consistencia** (datos desactualizados) que contradice el requisito de tiempo real, a cambio de un beneficio de rendimiento que aún no se necesita.

Preguntas abiertas E2

1. ¿Mantener enums para **vehicle_type**/categorías o migrar a tablas de catálogo si el negocio necesita agregarlos sin desplegar cambios de esquema?
2. ¿Cuándo conviene **particionar reservations por fecha** al crecer el historial, y qué política de retención/archivado aplica?
3. Dado que la base de datos PostgreSQL (RDS) y el backend se alojarán en subredes privadas (acorde a E1 en *Componentes del Curso*), ¿cuál será el mecanismo de red óptimo para que nuestro cómputo se comunique de forma segura con Amazon S3 para subir los PDFs de reservas?
 - a. ¿Utilizaremos un *Gateway VPC Endpoint* para evitar que el tráfico viaje por la internet pública o abriremos rutas de NAT Gateway?
4. Para cuando **Disponibilidad = 0** ¿Es más eficiente calcular el estado en tiempo real con *Lambda* periódica que publique una métrica customizada en *CloudWatch*, o deberíamos emitir **eventos** directamente desde la BD en cada mutación de reserva?

Anexo IA

1. Lo que le pedimos a la IA

- *"Actúa como un arquitecto de datos en la nube. Ayúdanos a estructurar una estrategia de persistencia para un sistema de reservas de parqueos corporativos de tamaño mediano (decenas de espacios). Compáranos el uso de un enfoque puramente relacional (PostgreSQL) contra uno NoSQL (DynamoDB), considerando la regla de negocio crítica: evitar double-booking y garantizar una sola reserva por día por usuario."*

2. Análisis y Reflexión del Equipo

- **Qué aceptamos:** En primera instancia la IA nos recomendó 100% una Base de datos relacional (ej. PostgreSQL). Evaluamos nuestra lógica de negocio, como por ejemplo *"permitir solo una reserva activa por usuario por día"* o *"validar si el espacio pertenece a la categoría asignada al colaborador"*. Estas requieren un aseguramiento en la integridad y consistencia en datos, además de manejar consistencia en datos, algo que con los principios ACID de una BD relacional nos queda excelente.
- **Qué editamos:**
- **Qué descartamos:** La IA sugirió implementar una capa de almacenamiento en caché en memoria distribuida usando *Redis (Amazon ElastiCache)* para acelerar el endpoint de disponibilidad de espacios (**GET /availability**), bajo el pretexto de que la lectura se actualiza cada 30 segundos. El equipo descartó la propuesta de inmediato para el MVP, debido a que el volumen de datos de un parqueo mediano no justifica la complejidad técnica extra de controlar la invalidación de la caché, pudiendo resolverse eficientemente con índices nativos en PostgreSQL sin generar

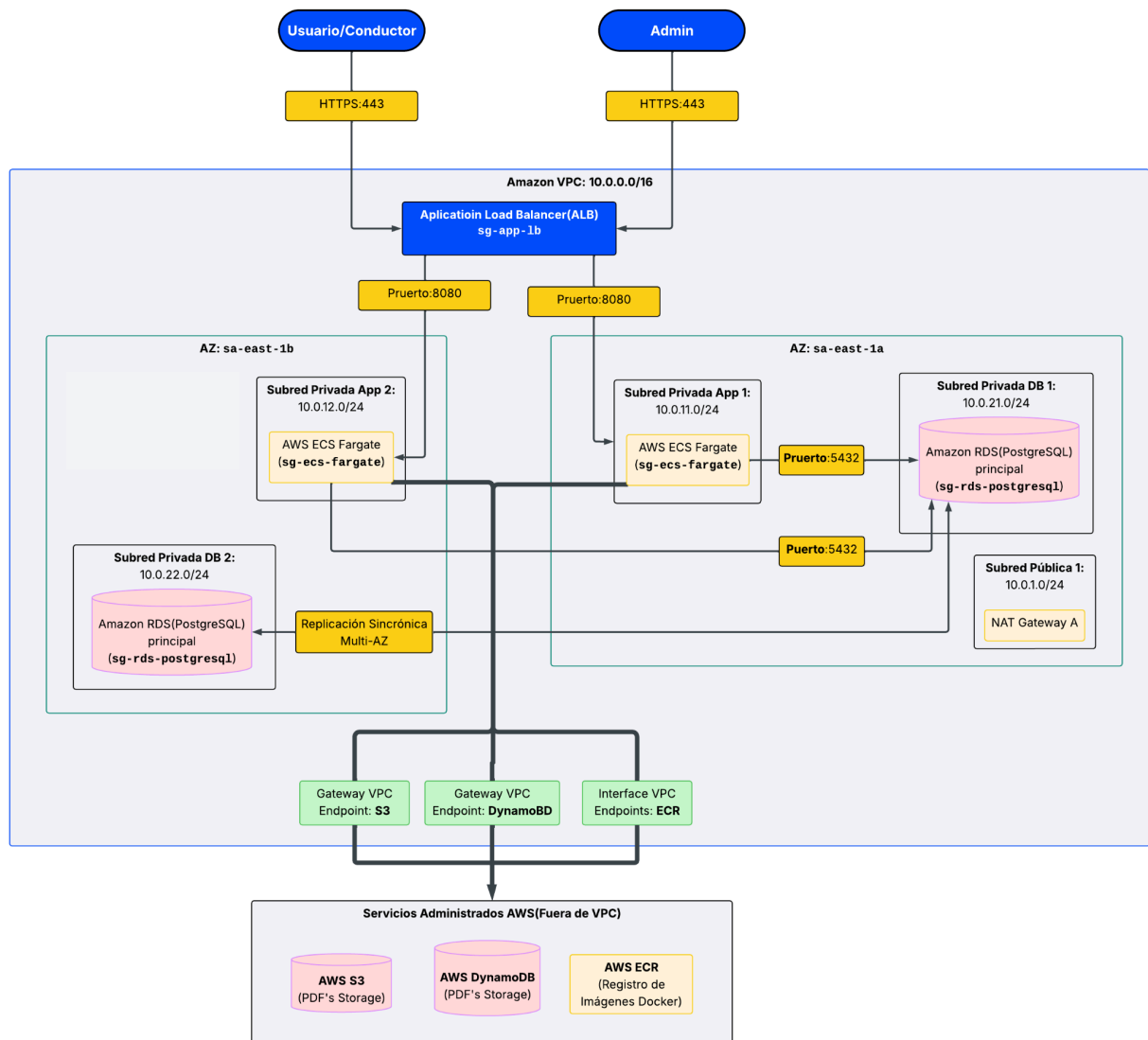
inconsistencias de datos en tiempo real. Añadir una caché ahora introduciría un problema de **invalidación y consistencia** (datos desactualizados) que **contradice el requisito de tiempo real**, a cambio de un beneficio de rendimiento que aún no se necesita.

ENTREGA 3

Resumen de cambios

- Nueva subsección: Decisión NAT Gateway vs. VPC Endpoints con análisis específico al sistema
- Nueva sección: Decisiones de Alcance LATAM (centralización VPC, AZs, región parametrizable)
- Adopción de Amazon ECR debido a ECS Fargate: Las tareas de ECS Fargate requieren comunicarse con Amazon ECR para descargar las imágenes de los contenedores de forma segura, ya sea a través del NAT Gateway o mediante un Interface VPC Endpoint dedicado (AWS PrivateLink).
- **Cambio arquitectónico:** región base us-west-2 → sa-east-1 como default LATAM. AZ naming actualizado. Es la única región AWS dentro del territorio latinoamericano, lo que la convierte en el argumento de residencia de datos más sólido frente a clientes corporativos LATAM que exigen que sus datos no salgan del continente.

Diagrama de contenedores (primera versión)



Diseño de red

Para garantizar la alta disponibilidad, la tolerancia a fallos y la seguridad del **Sistema de Reserva y Gestión de Parques Empresariales**, se ha diseñado una topología de red aislada utilizando una **Amazon VPC (Virtual Private Cloud)**. Este diseño mitiga los riesgos de exposición de datos sensibles de los colaboradores (como nombres, IDs y números de placas vehiculares) y asegura la resiliencia del sistema durante las horas pico de alta concurrencia (6:00 AM - 9:00 AM).

1. Direccionamiento IP y Segmentación de Subredes (Subnetting)

Se ha asignado a la VPC el bloque de direccionamiento principal **10.0.0.0/16**, lo que proporciona un espacio teórico de hasta 65,536 direcciones IP privadas, garantizando escalabilidad para futuras expansiones físicas del parqueo.

1.1 Justificación del Bloque 10.0.0.0/16 para Contexto LATAM

¿Por qué /16 y no un bloque más pequeño como /20 o /22?

La decisión no responde al número de colaboradores de la empresa cliente (que puede ser de 300 a 5,000 empleados para un parqueo formal en LATAM), sino a 2 criterios de ingeniería:

a) Dimensionamiento por servicios de infraestructura, no por usuarios

Las IPs de la VPC las consumen los componentes de infraestructura, no los usuarios finales del sistema. El inventario real por despliegue es el siguiente:

Componente	IPs consumidas	aproximadas
Tareas ECS Fargate activas (2–8 tasks en horario pico)		≤ 8
Instancias RDS PostgreSQL (primaria + réplica)		2
Nodos del ALB (gestionados por AWS)		2–6
NAT Gateway (1 por AZ)		1–2
Total operativo actual		≤ 18 IPs

Con subredes /24 (251 IPs utilizables cada una), el diseño actual ocupa 6 subredes × 251 = 1,506 IPs asignadas en el espacio de 65,536 (ocupación del 2.3%). Ese headroom no es desperdicio: es capacidad reservada para expansiones previsibles sin re-numeración.

b) Suficiencia para empresas LATAM con parqueos formales

Para una empresa con hasta 3,000 colaboradores con acceso al sistema de parqueos y un parqueo con 500 espacios, el sistema jamás necesitará más de ~50 IPs de infraestructura activas simultáneamente. El /16 es suficiente, y lo seguirá siendo incluso si el sistema expande a 10 empresas cliente en el mismo despliegue.

Para cumplir con las prácticas de **Alta Disponibilidad (HA)** sin triplicar los costos de transferencia de datos inter-zona (*Cross-AZ Data Transfer*), la infraestructura se distribuye simétricamente en **2 Zonas de Disponibilidad (Availability Zones: sa-east-1a y sa-east-1b)**. La red se segmenta verticalmente en una arquitectura de tres capas (*3-Tier Architecture*).

A. Capa Pública (DMZ, Demilitarized Zone)

Diseñada como barrera de seguridad entre la red interna de una organización y el Internet público.

- **Subred Pública 1 (AZ sa-east-1a):** 10.0.1.0/24 (251 IPs utilizables). Hospeda el nodo 'A' del Application Load Balancer (ALB) y el NAT Gateway de la zona 'a'.
- **Subred Pública 2 (AZ sa-east-1b):** 10.0.2.0/24 (251 IPs utilizables). Hospeda el nodo 'B' del ALB y provee redundancia de entrada pública.

B. Capa Privada de Cómputo (Application Layer)

Aislada del acceso directo a internet. Aquí reside la lógica de negocio del sistema.

- **Subred Privada App 1 (AZ sa-east-1a):** 10.0.11.0/24 (251 IPs utilizables). Hospeda los contenedores del API corriendo sobre **AWS ECS Fargate**.
- **Subred Privada App 2 (AZ sa-east-1b):** 10.0.12.0/24 (251 IPs utilizables). Hospeda las réplicas de los contenedores de ECS Fargate para balancear la carga activa.

C. Capa Privada de Datos (Data Layer)

Máximo nivel de aislamiento dentro de la VPC. No tiene rutas de salida hacia el internet.

- **Subred Privada Datos 1 (AZ sa-east-1a):** 10.0.21.0/24 (251 IPs utilizables). Hospeda la instancia primaria de **Amazon RDS PostgreSQL**.
- **Subred Privada Datos 2 (AZ sa-east-1b):** 10.0.22.0/24 (251 IPs utilizables). Hospeda la instancia secundaria/réplica de lectura de RDS PostgreSQL, configurada con despliegue Multi-AZ para conmutación por error (*failover*) automatizada.

2. Enrutamiento e Interconexión de Servicios Cloud

2.1 Flujo de Entrada (Inbound Routing):

1. El usuario (conductor) o el administrador realiza una petición HTTP/HTTPS al dominio del sistema.
2. La solicitud llega al **Internet Gateway (IGW)** asociado a la VPC, el cual la deriva al **Application Load Balancer (ALB)** distribuido en las subredes públicas.
3. El ALB actúa como proxy inverso: termina la conexión SSL/TLS y distribuye el tráfico balanceado hacia las tareas de **AWS ECS Fargate** en las subredes privadas de cómputo a través del puerto interno 8080.

2.2 Flujo de Salida y Optimización de Costos (Outbound Routing):

- Decisión

Los contenedores de ECS Fargate requieren salida controlada a internet para tres operaciones críticas del sistema: (1) validación de tokens contra Autho, (2) despacho de correos con comprobante PDF a través de SendGrid/SES, y (3) descarga de imágenes de contenedores desde Amazon ECR durante el arranque o escalado. Adicionalmente, el sistema sube comprobantes en PDF/QR a Amazon S3 y escribe registros de auditoría en DynamoDB.

La arquitectura implementa una **estrategia híbrida** con dos rutas de tráfico diferenciadas:

- **Ruta A — NAT Gateway** (0.0.0.0/0 en tablas de enrutamiento privadas): Para tráfico con destino a servicios de terceros externos (Autho, SendGrid) donde no existe alternativa de VPC Endpoint.
- **Ruta B — Gateway VPC Endpoints** (S3 y DynamoDB): Para tráfico con destino a servicios internos de AWS, que se desvía completamente del NAT Gateway y viaja por la red troncal privada de AWS.

- Trade-offs

- **Trade-off 1: NAT Gateway único (AZ-a) vs. NAT Gateway redundante (una instancia por AZ)**

El diseño actual despliega un único NAT Gateway en `sa-east-1a`. Las tareas ECS en `sa-east-1b` enrutan su salida hacia ese NAT en zona cruzada.

Dimensión		Opción actual: NAT único en AZ-a	Alternativa: NAT por AZ
Costo	fijo mensual	~\$32.4/mes (1 gateway × \$0.045/hr × 720h)	~\$64.8/mes (2 gateways)
Resiliencia de salida		Si <code>sa-east-1a</code> falla, los contenedores en <code>sa-east-1b</code> pierden capacidad de llamar a Autho y SendGrid	Cada AZ opera de forma autónoma; la caída de una zona no afecta la salida de la otra

Impacto en el sistema de parqueos	Durante el horario pico 06:00–09:00, un fallo de <code>sa-east-1a</code> impide que ECS en <code>sa-east-1b</code> valide tokens: el sistema queda parcialmente inoperante aunque la base de datos siga activa	Mantiene la disponibilidad completa del flujo de reservas aunque caiga una AZ
Decisión para el proyecto	Aceptado para MVP; el Cross-AZ data transfer (\$0.01/GB) en tráfico de notificaciones es mínimo (~15 MB/día estimado). En producción con SLA formal, se añade el segundo NAT.	Recomendado cuando el sistema tenga SLA contractual con empresas cliente

La postura actual refleja que el MVP opera sin SLA contractual. Al formalizar el servicio con clientes LATAM, el costo adicional de ~\$32/mes se justifica por mantener el flujo de autenticación y notificaciones resiliente ante fallas de AZ, que según AWS tienen una frecuencia histórica de ~2–4 eventos de degradación por año por AZ.

- **Trade-off 2: Gateway VPC Endpoints (S3 + DynamoDB) vs. enrutar por NAT Gateway**

Dimensión	Gateway VPC Endpoint	Ruta por NAT
Costo de datos	\$0.00 por GB transferido	\$0.045/GB procesado por el NAT
Latencia de upload (S3)	Tráfico en red interna de AWS; evita saltos de internet (~30–80ms de diferencia en round-trip)	El PDF del comprobante (~150 KB) transita por internet antes de volver a AWS
Superficie de ataque	Los datos de comprobantes (que vinculan nombre, placa e ID de reserva) nunca salen de la red de AWS	Cualquier ruta por internet amplía el perímetro de inspección
Complejidad operativa	Requiere gestionar entradas adicionales en tablas de enrutamiento (2 entradas prefix-list por endpoint)	Sin configuración adicional
Portabilidad Terraform	Se declara como recurso <code>aws_vpc_endpoint</code> en el módulo de red; exportado como output para consumo del módulo de cómputo (principio Sesión 6: módulos consumen outputs, nunca IDs literales)	No aplica

3. Estrategia de Firewalls: Grupos de Seguridad (Security Groups)

Se aplica de forma estricta el principio de mínimo privilegio mediante reglas encadenadas por referencia de ID de Security Group (no por rangos CIDR), siguiendo el siguiente principio:

"Las reglas deben referenciar objetos de Security Group río arriba, no rangos CIDR, para que las reglas sigan al objeto y no a la IP."

Esta práctica es especialmente crítica en entornos ECS Fargate donde las IPs de los contenedores cambian dinámicamente en cada tarea.

1. **sg-application-lb (Asociado al ALB):**
 - **Ingreso (Inbound):** Permite puertos 80 (HTTP) y 443 (HTTPS) desde cualquier origen (0.0.0.0/0).
 - **Egreso (Outbound):** Permite tráfico únicamente hacia el puerto 8080 apuntando explícitamente al identificador sg-ecs-fargate.
2. **sg-ecs-fargate (Asociado a los contenedores de la API):**
 - **Ingreso (Inbound):** Permite tráfico en el puerto 8080 **únicamente** si la fuente proviene del grupo sg-application-lb. Bloquea cualquier petición directa de internet.
 - **Egreso (Outbound):** Permite tráfico total (0.0.0.0/0) para conectarse con los endpoints externos de autenticación, el NAT Gateway y los VPC Endpoints.
3. **sg-rds-postgresql (Asociado a la Base de Datos):**
 - **Ingreso (Inbound):** Permite conexiones en el puerto 5432 **únicamente** si provienen del grupo de seguridad de cómputo sg-ecs-fargate. Se deniega el acceso incluso a las subredes públicas.
 - **Egreso (Outbound):** Denegado por defecto (No requiere iniciar conexiones salientes).

Preguntas abiertas

- ¿Qué pasaría si expandimos las zonas de disponibilidad? ¿Qué pasaría si nuestro sistema se expande y llega a continentes europeos o asiáticos? ¿Qué zonas habríamos de disponer y cómo manejamos los costos?
- Dado que el sistema está centralizado en una única VPC en la región **sa-east-1** (São Paulo) para dar servicio a múltiples países de LATAM, ¿cómo afectaría la latencia de red acumulada y un eventual corte de fibra submarina internacional al SLA del Sistema (< 30 segundos)? ¿Vale la pena diseñar un esquema de replicación asincrónica de la base de datos RDS hacia una región secundaria con costos de transferencia transcontinental reducidos (ej. **us-east-1**), o el riesgo se mitiga delegando la resiliencia puramente a las zonas de disponibilidad locales (**sa-east-1a** y **sa-east-1b**)?

- Al procesarse miles de reservas diarias por cada sede corporativa, ¿en qué momento exacto del tiempo es financieramente óptimo realizar la transición de los archivos desde *S3 Standard* hacia *S3 Glacier Flexible Retrieval* o *Deep Archive*?
- Considerando que nuestro público objetivo se de LATAM, también encuentra en México y Centroamérica, el diseño actual centralizado en EE. UU. introduce una latencia base debido a la distancia geográfica. ¿Es justificable mitigar esta latencia agregando una tercera Zona de Disponibilidad local para absorber ráfagas de concurrencia y caídas de datacenters, o el problema se debe atacar implementando una capa perimetral como *Amazon CloudFront* o *AWS Global Accelerator*? ¿Cómo balanceamos el costo fijo de añadir NAT Gateways y transferencia de datos inter-AZ adicionales frente al costo variable por transferencia de datos saliente (*Data Transfer Out*) que cobra una red de distribución de contenido (CDN)?

Anexo IA

1. Qué se le pidió a la IA

Se le solicitó una estructura de subredes con máscaras CIDR optimizadas para una arquitectura de tres capas en AWS utilizando ECS Fargate y RDS, buscando alta disponibilidad en 2 AZs convenientes para países de LATAM.

2. Análisis y Reflexión del Equipo

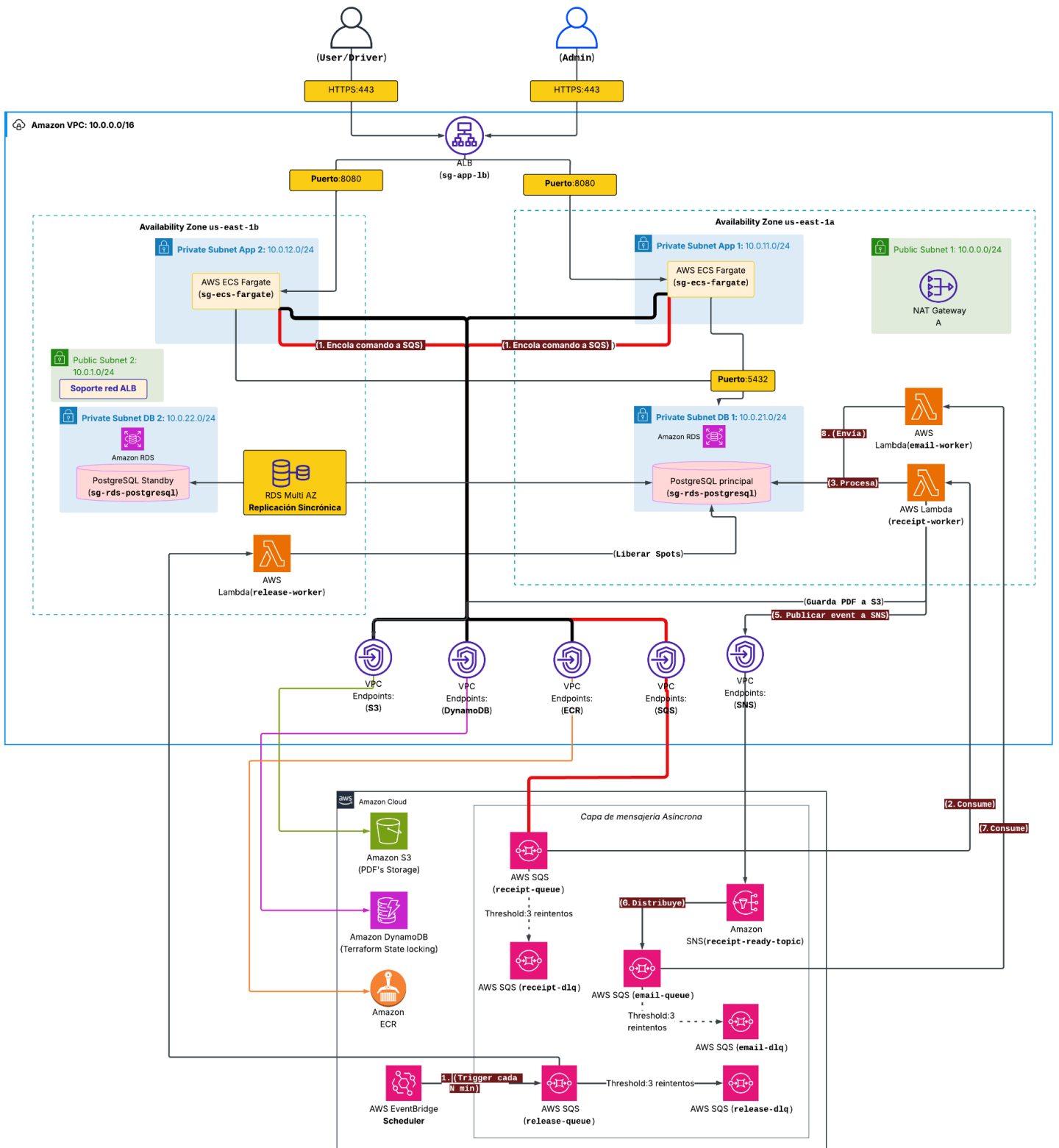
- **Qué se aceptó:** El direccionamiento propuesto basado en bloques `/24` para cada subred, ya que provee un balance adecuado entre el aislamiento de entornos (251 IPs utilizables por zona) y evita el desperdicio excesivo de direccionamiento en la VPC principal `/16`.
 - Se aceptó el manejo de las subredes en `sa-east-1` para garantizar menor latencia entre la mayoría de países de LATAM, aunque si queremos expandirnos más, podríamos aceptar uno o más en EU para asegurar mejor latencia en países como MX y centroamérica; claro que implican más costos y hay mejores opciones como un CDN.
- **Qué se editó:** La IA sugería colocar las tablas de enrutamiento públicas de manera unificada. El equipo editó la propuesta separando las tablas de enrutamiento públicas y privadas de manera independiente por cada Zona de Disponibilidad, garantizando que el fallo de un NAT Gateway en la AZ `sa-east-1a` no bote la conectividad saliente de la AZ `sa-east-1b`.
- **Qué se descartó:** La recomendación automatizada de utilizar un *AWS Transit Gateway* y *VPC Peering* para conectar los microservicios. Se descartó inmediatamente porque el alcance transaccional actual del sistema de parqueos se resuelve de forma centralizada en una única VPC monolítica de red, evitando costos innecesarios de ruteo inter-VPC.
 - Además descartamos las residencias principales en estados unidos(donde se encuentran las mayorías de datacenters).

ENTREGA 4

Resumen de cambios

- La generación del comprobante y la liberación automática de reservas se sacaron del contenedor del API y ahora corren como workers asíncronos vía SQS y SNS, porque antes, si el contenedor se reiniciaba, el trabajo se perdía sin reintento.
- Se agregaron a la arquitectura las colas SQS con sus Dead Letter Queues, el tópico SNS, los workers en Lambda y el disparador EventBridge Scheduler para la liberación periódica.
- La API se mantiene en Fargate y solo los workers asíncronos se implementan en Lambda, por ser procesos cortos disparados por eventos.
- Se aclaró que las colas no intervienen en la doble reserva: la atomicidad sigue garantizada por los índices únicos parciales en PostgreSQL de la Entrega 2.
- El control de idempotencia se resuelve con una tabla en PostgreSQL en lugar de Redis, en coherencia con la Entrega 2 que ya había descartado ElastiCache por el bajo volumen.
- Se prevén VPC Endpoints (PrivateLink) para SQS y SNS en las subredes privadas, alineado con los endpoints que la Entrega 3 ya usaba para S3 y DynamoDB.
- **EventBridge Scheduler como disparador periódico:** Se incorporó Amazon EventBridge Scheduler para ejecutar periódicamente el barrido de reservas expiradas, reemplazando el timer interno dentro del contenedor Fargate que se perdía al reiniciar la tarea.
- Se cambiaron las regiones de `sa-east-1` a `us-east-1` dado que si bien es cierto nuestro público objetivo es LATAM, es muy posible que personas del extranjero utilicen (por ejemplo E.U.) la app ya que no hay motivos de restricciones (de nuestro sistema) en cuanto a países con la disponibilidad de usar el sistema. Aunque si no fuera así, se seguiría manteniendo el sistema con pocas regiones y usando simplemente CDN ya que nuestra clase de sistema no es tan dinámico y tan personalizado.

Diagrama de contenedores actualizado



Resumen de cambios

- Migración de región: Se actualizó la región base de sa-east-1 a us-east-1 en todas las AZs, VPC Endpoints y nomenclatura de recursos, según el lineamiento del proyecto.
- **Adición de VPC Endpoints para SQS y SNS (PrivateLink):** Se incorporaron Interface VPC Endpoints (com.amazonaws.us-east-1.sqs, com.amazonaws.us-east-1.sns) en las subredes privadas, siguiendo el mismo patrón de la Entrega 3 para ECR. Esto asegura que el tráfico de mensajería asíncrona nunca salga a internet y no incurra en costos de NAT Gateway.
- **Workers asíncronos en AWS Lambda (VPC-enabled):** Se trasladaron los procesos de generación de comprobantes, liberación de reservas y envío de correos a funciones Lambda en lugar de ejecutarse dentro del contenedor Fargate. Lambda es el modelo correcto para procesos cortos y orientados a eventos, mientras que Fargate continúa alojando la API REST persistente (consistente con la decisión de la Entrega 2). Las funciones Lambda se configuran con VPC-attach para acceder a RDS desde las subredes privadas.
- **Arquitectura de colas SQS con Dead Letter Queues:** Se diseñaron tres colas principales (receipt-queue, release-queue, receipt-email-queue) con sus respectivas DLQ, cada una configurada con maxReceiveCount = 3 y backoff exponencial. Los mensajes fallidos se conservan íntegros en las DLQ para auditoría y reinyección manual.
- **Tópico SNS con fan-out hacia SQS:** El evento ReceiptReadyEvent se publica en un tópico SNS que suscribe a receipt-email-queue (cola SQS), en lugar de suscribir Lambda directamente. Esto hereda el mecanismo de reintentos y DLQ de SQS para el envío de correos.
- Se justifica la integración de **Amazon EventBridge Scheduler** como el componente encargado de disparar de forma periódica (cada N minutos) el proceso de liberación de espacios de parqueo expirados. Su uso mitiga el antipatrón de mantener temporizadores en memoria dentro de los contenedores efímeros de ECS Fargate (Express.js), los cuales generarían ejecuciones duplicadas al escalar horizontalmente o pérdida de tareas programadas ante reinicios de los contenedores. EventBridge actúa de forma serverless y desacoplada, garantizando un mecanismo confiable de alta disponibilidad para la consistencia del inventario de parqueos sin incurrir en costos de cómputo ocioso.

Flujos asíncronos

1. Distinción entre Comando y Evento

El diseño asíncrono del sistema se apoya en dos tipos de mensaje, y conviene definirlos con claridad porque se usan en situaciones distintas.

Un comando es una instrucción para que algo se ejecute. Describe una acción pendiente, tiene un único destinatario y se espera que se procese una sola vez. Los comandos viajan por Amazon SQS, que garantiza la entrega del mensaje y permite reintentarlo si el procesamiento falla.

Un evento es la notificación de algo que ya ocurrió. No le ordena nada a nadie en particular, solo anuncia un cambio de estado para que los interesados reaccionen por su cuenta. Un mismo evento puede tener varios destinatarios. Los eventos viajan por Amazon SNS, que distribuye una copia del mensaje a cada suscriptor (patrón fan-out).

La diferencia se resume así: el comando se nombra en imperativo y representa trabajo por hacer; el evento se nombra en pasado y representa trabajo ya realizado.

Cómputo de los workers

La API REST seguirá ejecutándose sobre ECS Fargate, como se decidió en la Entrega 2, porque es un servidor HTTP persistente que debe responder de forma continua. Los workers asíncronos, en cambio, se ejecutarán como funciones AWS Lambda: son procesos cortos, disparados por la llegada de un mensaje, que no requieren un servidor corriendo de forma permanente.

Los tres flujos asíncronos del sistema

Del análisis del backend actual en Express.js surgen tres procesos que conviene volver asíncronos:

- a) **GenerateReceiptCommand (comando):** Cuando **POST /reservar** confirma la reserva y responde HTTP 201, en lugar de generar el comprobante en el mismo proceso, el API publica este comando en una cola SQS. Un worker lo consume, genera el PDF con código QR, lo sube a Amazon S3 y actualiza el campo **receipt_s3_key** de la reserva. Es una relación uno a uno: un comando produce un comprobante.
- b) **ReceiptReadyEvent (evento):** Cuando el comprobante queda guardado en S3, el worker publica este evento en un tópico SNS para anunciar que la reserva está lista. Varios suscriptores reaccionan de forma independiente: el envío del correo con el comprobante al conductor (requisito del caso de uso 3: comprobante por correo en menos de 30 segundos) y, como extensión futura, la actualización del dashboard de ocupación del administrador. Es una relación uno a muchos.
- c) **ReleaseExpiredReservationCommand (comando):** La liberación automática de reservas no confirmadas a los 20 minutos deja de ejecutarse como una tarea programada dentro del contenedor del API y pasa a dispararse con Amazon EventBridge Scheduler, que coloca periódicamente este comando en SQS para que un worker libere las reservas vencidas.

Productores y consumidores

Mensaje	Transporte	Productor	Consumidor
GenerateReceiptCommand	Amazon SQS	API Express.js	Worker de comprobantes
ReceiptReadyEvent	Amazon SNS	Worker de comprobantes	Servicio de correo y dashboard

ReleaseExpiredReservationCommand	EventBridge Scheduler → SQS	EventBridge Scheduler	Worker de liberación
----------------------------------	-----------------------------	-----------------------	----------------------

2. Definición de Payloads

Cada mensaje transporta un objeto JSON con dos partes: un encabezado de control común a todos los mensajes y un bloque **data** con la información de negocio específica. Esta separación permite que los workers procesen el control del mensaje (identidad, tipo, momento de emisión) sin necesidad de interpretar el contenido de negocio.

Campos de control comunes

Los tres mensajes incluyen los siguientes campos en el nivel superior:

Campo	Tipo	Propósito
event_type	string	Nombre del mensaje (GenerateReceiptCommand, ReceiptReadyEvent, ReleaseExpiredReservationCommand). Permite distinguir y enrutar el mensaje.
idempotency_key	string (UUID)	Identificador único del mensaje. Es la base del control de duplicados descrito en la sección de idempotencia.
timestamp	string (ISO 8601)	Momento de emisión del mensaje. Se usa para auditoría, ordenamiento y para medir la latencia frente al SLA de 30 segundos del comprobante.

Decisión de seguridad

Los mensajes transportan únicamente identificadores (**reservation_id**, **user_id**, **space_id**). Datos personales como el nombre del conductor o el número de placa no se incluyen en el payload. El worker que necesita esa información la obtiene directamente de la base de datos: lee **users.full_name** y descifra **vehicles.plate_enc**, igual que lo hace el backend actual. De este modo, ningún dato sensible circula por las colas ni por los tópicos, manteniendo la coherencia con la política de cifrado de datos definida desde la Entrega 1. Como consecuencia de diseño, el worker de comprobantes requiere conexión a la base de datos RDS.

Payload 1: GenerateReceiptCommand

Lo produce el endpoint **POST /reservar** tras confirmar la reserva; lo consume el worker de comprobantes.

Los campos del bloque data corresponden a los valores que el backend ya maneja en la respuesta 201 de **POST /reservar**: **reservation_id** es el identificador generado por el

INSERT, y `space_id`, `vehicle_id` y `reservation_date` provienen de la misma respuesta. Con estos identificadores el worker recupera de la base de datos el nombre del conductor y la placa, genera el PDF con código QR, lo sube a Amazon S3 y actualiza `reservations.receipt_s3_key`.

```
{
  "event_type": "GenerateReceiptCommand",
  "idempotency_key": "a3f1c8e2-5b7d-4e9a-8c1f-2d6b4a9e0f3c",
  "timestamp": "2026-05-17T08:30:00Z",
  "data": {
    "reservation_id": 4042,
    "user_id": 87,
    "space_id": 12,
    "vehicle_id": 5,
    "reservation_date": "2026-05-17"
  }
}
```

Payload 2: ReceiptReadyEvent

Lo produce el worker de comprobantes cuando el PDF queda almacenado en S3; lo consumen el servicio de correo y, como extensión futura, el dashboard del administrador.

El campo `receipt_s3_key` sigue el formato que el backend ya utiliza (`receipts/<id>.pdf`). El suscriptor de correo usa `user_id` para localizar la dirección del conductor y `receipt_s3_key` para generar una URL prefirmada y adjuntar el comprobante. El evento se mantiene ligero de forma intencional, solo anuncia que el comprobante de una reserva está disponible, y cada suscriptor decide cómo reaccionar.

```
{
  "event_type": "ReceiptReadyEvent",
  "idempotency_key": "b7e2d9f4-1a3c-4f8b-9d2e-6c5a8b1f0e4d",
  "timestamp": "2026-05-17T08:30:04Z",
  "data": {
    "reservation_id": 4042,
    "user_id": 87,
    "receipt_s3_key": "receipts/4042.pdf",
  }
}
```

Payload 3: ReleaseExpiredReservationCommand

Lo produce Amazon EventBridge Scheduler de forma periódica; lo consume el worker de liberación.

A diferencia de los otros dos mensajes, este no opera sobre una reserva específica, sino que dispara un barrido. El campo **scan_before** indica al worker el instante de corte: debe liberar toda reserva en estado reservada cuyo **confirm_deadline** sea anterior a ese momento, cambiándolas al estado expirada. Este comportamiento corresponde directamente a la consulta que el worker actual ya ejecuta sobre el índice.

```
{
  "event_type": "ReleaseExpiredReservationCommand",
  "idempotency_key": "c1d4e7a0-9f2b-4c6d-8e3a-1b5f7d9c2a8e",
  "timestamp": "2026-05-17T08:50:00Z",
  "data": {
    "scan_before": "2026-05-17T08:50:00Z"
  }
}
```

3. Estrategia de Idempotencia

En un sistema de mensajería asíncrona, un mismo mensaje puede llegar más de una vez. Amazon SQS garantiza entrega al menos una vez (at-least-once), lo que significa que, ante reintentos o demoras en el procesamiento, la cola puede entregar un duplicado. La idempotencia es la propiedad que asegura que procesar el mismo mensaje dos veces produzca el mismo resultado que procesarlo una sola vez. Cada worker debe protegerse contra duplicados.

El sistema aplica tres estrategias, una por flujo, todas resueltas en la base de datos sin introducir servicios adicionales.

a) Flujo 1: GenerateReceiptCommand

Antes de generar el comprobante, el worker consulta si la reserva ya tiene poblado el campo **receipt_s3_key**. Si ya contiene una clave, significa que un mensaje anterior generó el comprobante, por lo que el worker descarta el duplicado sin realizar trabajo. Si el campo está vacío (NULL), procede a generar el PDF, subirlo a S3 y poblar el campo. Como refuerzo, el archivo en S3 se nombra con una clave determinista (**receipts/<id>.pdf**). Aun si dos ejecuciones llegaran a coincidir, ambas escribirían el mismo objeto con contenido idéntico, sin generar comprobantes duplicados.

b) Flujo 2: ReceiptReadyEvent

Este es el único flujo que requiere un mecanismo explícito, porque el envío de un correo no es idempotente por sí mismo: enviarlo dos veces haría que el conductor

reciba dos correos. Para evitarlo, el suscriptor de correo mantiene una tabla de control en PostgreSQL (**processed_messages**) que usa la **idempotency_key** como llave primaria. Antes de enviar el correo, el suscriptor intenta insertar la clave del mensaje; si la inserción falla por violación de unicidad, el mensaje ya fue procesado y el envío se omite. Este mecanismo reutiliza el mismo patrón de índice único que el sistema ya emplea para garantizar la atomicidad de las reservas, aplicando una técnica que el proyecto ya domina en lugar de introducir una nueva.

c) Flujo 3: **ReleaseExpiredReservationCommand**

Este flujo es idempotente por la naturaleza de su operación. El worker ejecuta una actualización condicionada al estado actual de la reserva:

```
UPDATE reservations
SET status = 'expirada'
WHERE status = 'reservada' AND confirm_deadline <
:scan_before
```

Si el mensaje se procesara dos veces, la segunda ejecución no encontraría filas que cumplan la condición, ya que las reservas afectadas dejaron de estar en estado reservada. La cláusula **WHERE status = 'reservada'** actúa como el mecanismo de idempotencia: reprocesar el mensaje no produce ningún efecto adicional.

PostgreSQL en lugar de un servicio de caché dedicado

El control de idempotencia se resuelve en la base de datos PostgreSQL existente y no en un servicio de caché en memoria como Amazon ElastiCache (Redis). Esta decisión es coherente con la Entrega 2, donde se descartó introducir Redis para el MVP por considerar que el volumen del sistema —decenas de espacios, reservas concentradas en ventanas predecibles— no justifica la complejidad operativa de un servicio adicional. El volumen de mensajes asíncronos es igualmente bajo, por lo que una tabla de control en la base de datos es suficiente y evita añadir un componente nuevo que administrar.

4. Manejo de Fallos y DLQ

El backend actual maneja los fallos de la peor manera posible para un proceso crítico: cuando la generación del comprobante falla, el error solo se registra en el log y se descarta, sin reintento ni recuperación. Lo mismo ocurre en el worker de liberación. El comprobante simplemente nunca se crea y nadie se entera. Trasladar estos procesos a SQS resuelve precisamente este problema: convierte un fallo que se pierde en un log en un fallo que se reintenta automáticamente y que, de no poder resolverse, se preserva para auditoría.

Reintentos con backoff exponencial

Cuando un worker no logra procesar un mensaje ya sea por una falla de generación del PDF, una indisponibilidad momentánea de S3 o de la base de datos, SQS no descarta el mensaje, sino que lo vuelve a poner disponible para reintentarlo. Entre un intento y el siguiente, el tiempo de espera crece de forma exponencial (backoff exponencial), implementado mediante

el incremento progresivo del visibility timeout del mensaje. Esto evita saturar un servicio que está temporalmente caído y le da tiempo a recuperarse.

Threshold de reintentos

Cada cola principal se configura con un máximo de 3 intentos de procesamiento, definido por el parámetro **maxReceiveCount** en la política de redrive de SQS. Tres intentos son suficientes para absorber fallos transitorios (un timeout de red, un reinicio breve de la base de datos) sin insistir indefinidamente en un mensaje que es irrecuperable. Una vez superado ese umbral, el mensaje se traslada automáticamente a su Dead Letter Queue.

Dead Letter Queue por flujo

Cada cola principal tiene su propia Dead Letter Queue (DLQ), lo que permite diagnosticar cada flujo de forma independiente. Los nombres siguen la convención de recursos del proyecto (<nombre>-<entorno>-<recurso>):

- El flujo de comprobantes utiliza la cola <nombre>-<entorno>-receipt-queue, respaldada por su cola de mensajes fallidos <nombre>-<entorno>-receipt-dlq. Un mensaje que llega a esta DLQ indica que el comprobante no pudo generarse tras tres intentos, de modo que la reserva existe pero quedó sin comprobante y requiere intervención manual.
- El flujo de liberación utiliza la cola <nombre>-<entorno>-release-queue y su correspondiente <nombre>-<entorno>-release-dlq. Un mensaje en esta DLQ indica que el barrido de liberación falló de forma persistente, lo que significa que las reservas vencidas no llegaron a liberarse y el equipo debe revisarlas.
- Finalmente, el envío de correo utiliza la cola <nombre>-<entorno>-receipt-email-queue y su <nombre>-<entorno>-receipt-email-dlq. Un mensaje en esta DLQ indica que el correo con el comprobante no pudo entregarse tras tres intentos.

Los mensajes que llegan a una DLQ no se procesan automáticamente: quedan almacenados íntegros para inspección y auditoría manual. El equipo puede examinarlos, identificar la causa raíz del fallo, corregirla y, si corresponde, reinyectarlos a la cola principal (operación de redrive). Este es exactamente el comportamiento que hoy no existe: en lugar de un error perdido en un log, el mensaje fallido se conserva completo.

Manejo de fallos en el evento SNS

A diferencia de SQS, Amazon SNS no ofrece por sí mismo un mecanismo de reintentos con DLQ equivalente. Por ello, el suscriptor de correo, que necesita garantías de entrega, no se suscribe directamente al tópico SNS, sino que lo hace a través de su propia cola SQS (<nombre>-<entorno>-receipt-email-queue). De este modo, el envío del correo hereda el mismo mecanismo de reintentos y DLQ que el resto de los flujos: el tópico **ReceiptReadyEvent** publica en esa cola, y el worker de correo la consume con la misma política de 3 intentos y backoff exponencial.

Relación con la idempotencia

El mecanismo de reintentos hace que un mensaje pueda procesarse más de una vez, lo que vuelve indispensable la estrategia de idempotencia descrita en la sección anterior. Ambos mecanismos trabajan en conjunto: los reintentos garantizan que el trabajo llegue a completarse, y la idempotencia garantiza que reprocesar un mensaje no produzca efectos duplicados.

Preguntas abiertas

1. ¿Qué métricas y alarmas mínimas necesitamos sobre las colas para operar con confianza? En particular, ¿conviene alarmar sobre la antigüedad del mensaje más viejo (**ApproximateAgeOfOldestMessage**) en la cola de comprobantes para detectar que el SLA de 30 segundos se está incumpliendo, o basta con alarmar cuando lleguen mensajes a las DLQ? ¿Cuál es el umbral razonable para cada una?
2. Definimos que los datos personales no viajan en el payload y que los workers leen de RDS, pero queda por detallar el acceso de red: ¿los workers Lambda se colocan dentro de la VPC para alcanzar RDS y los VPC Endpoints de SQS/SNS, asumiendo el costo y la latencia de arranque que eso implica, o existe una alternativa? ¿Qué política IAM de mínimo privilegio aplica a cada worker (qué cola puede leer, en qué tópico puede publicar)?
3. La tabla **processed_messages** en PostgreSQL crece de forma indefinida. ¿Qué política de retención o limpieza aplicamos para que no degrade la base de datos con el tiempo, y a partir de qué volumen de mensajes la validación de idempotencia en PostgreSQL dejaría de ser suficiente frente a una solución como Redis con expiración por TTL?

Anexo IA

1. Lo que le pedimos a la IA

Durante el diseño de los flujos asíncronos consultamos a la IA en tres frentes:

Sobre la distinción comando/evento

Actúa como arquitecto de soluciones en AWS. Tengo un backend Express.js que hoy genera un comprobante PDF de forma síncrona tras crear una reserva, y un worker de liberación que corre como cron dentro del mismo contenedor. Quiero pasar esto a un patrón asíncrono. Ayúdame a distinguir qué partes deben modelarse como comando (SQS) y cuáles como evento (SNS), y por qué.

Sobre la idempotencia

En un sistema de mensajería con SQS que garantiza entrega at-least-once, ¿cómo garantizo idempotencia en los workers? Compárame validar las claves contra Redis (ElastiCache) frente a usar la base de datos relacional que ya tengo (PostgreSQL), para un sistema de bajo volumen.

Sobre el manejo de fallos

Para mis colas SQS, ¿qué threshold de reintentos y qué política de Dead Letter Queue es razonable? Mi notificación va por SNS: ¿cómo le doy reintentos y DLQ a un suscriptor de SNS?

2. Análisis y Reflexión del Equipo

Durante el diseño de los flujos asíncronos se tuvo la siguiente reflexión:

Sobre la distinción comando/evento

- **Qué aceptamos:** La distinción comando/evento y modelar el comprobante como comando y la notificación como evento.
- **Qué editamos:** Los nombres de los mensajes, para que coincidieran con identificadores ya existentes en el código.
- **Qué descartamos:** Encolar la reserva para evitar doble reserva, por contradecir la atomicidad por índices únicos de la Entrega 2.

Sobre la idempotencia

- **Qué aceptamos:** El concepto de clave de idempotencia y que un UPDATE condicionado al estado es idempotente por naturaleza.
- **Qué editamos:** Aplicamos una estrategia distinta por flujo en lugar de una sola técnica uniforme.
- **Qué descartamos:** Usar Redis (ElastiCache) para las claves, por coherencia con la Entrega 2; queda como evolución futura.

Sobre el manejo de fallos

- **Qué aceptamos:** El threshold de 3 reintentos con backoff exponencial y el patrón SNS→SQS para el suscriptor de correo.
- **Qué editamos:** Una DLQ independiente por flujo, alineada a la convención de nombres de nuestro Terraform.